

Role of Assertions in Unit Testing

Understanding how **assertions** verify your **code** and determine test outcomes

 by Rodrigo Martins Pagliares

Last update: 11/25/2025

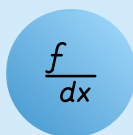


Role of Assertions in Unit Testing



Verify Expected Output

Assertions **verify** whether the **method** under test **produces** the **expected output**



Evaluate Conditions

Each **assertion evaluates** a **condition** based on supplied **input values**



Handle Failures

If the evaluated **condition** is **false**, the **assertion fails** and **throws** an **exception**

Assertion failures indicate that the unit test should fail

Assertions are the **core mechanism** that determines **pass/fail** outcomes in **tests**

Adding Custom Messages to Assertions



01

Optional Custom Messages

Assertions can accept an optional custom message to describe failure context

02

Best Practice

Good practice: always include an informative and descriptive error message

03

Failure-Only Display

Messages appear **only when** the test fails, helping identify the issue quickly

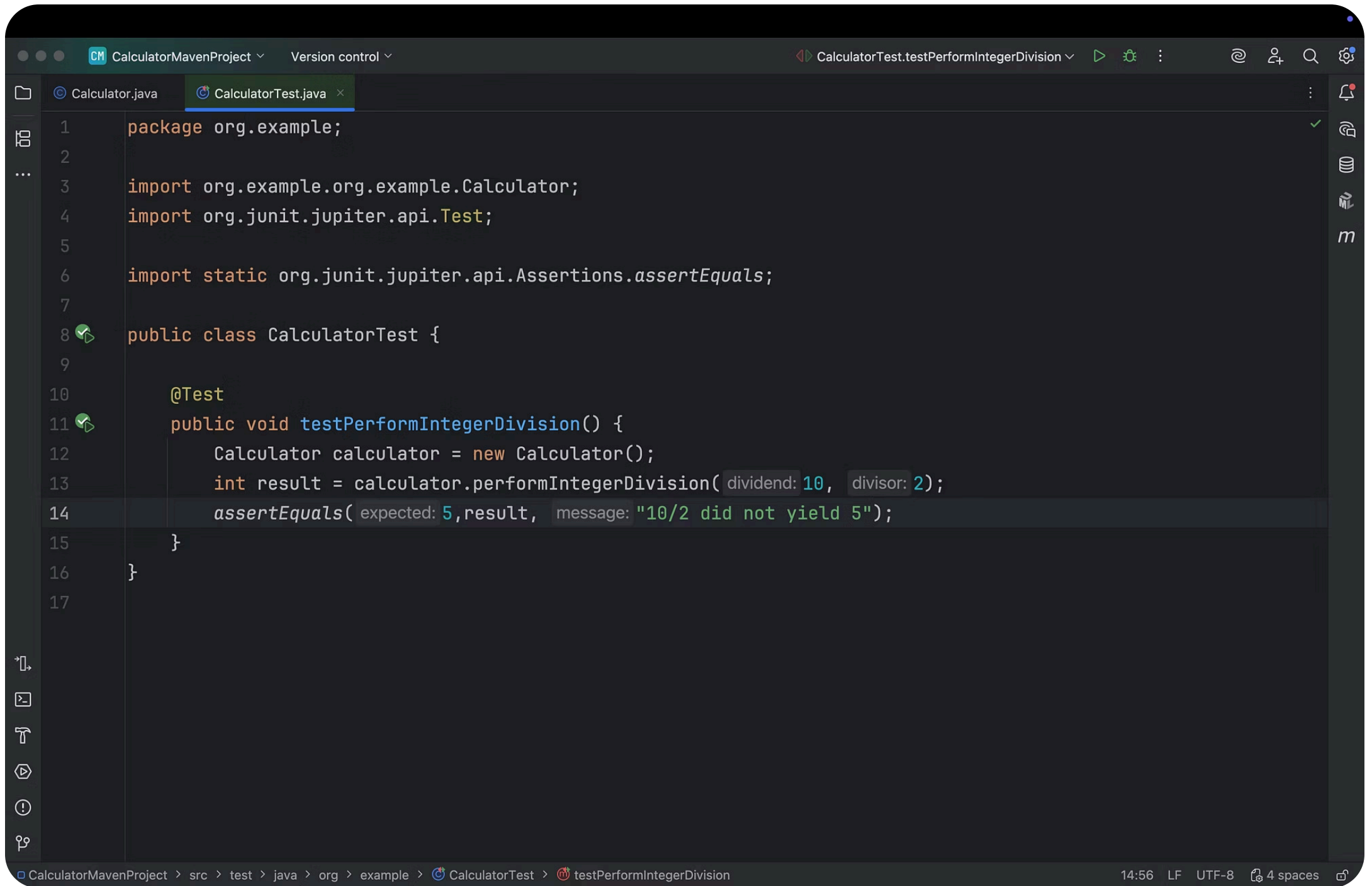
04

Team Communication

Clear messages help teammates understand what went wrong without inspecting the code

 **Example:** "10 divided by 2 did not yield 5" provides immediate clarity in reports

Adding Custom Messages to Assertions



The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The active file is 'CalculatorTest.java'. The code defines a 'CalculatorTest' class with a test method 'testPerformIntegerDivision()'. Inside this method, a 'Calculator' object is instantiated, and the 'performIntegerDivision()' method is called with a dividend of 10 and a divisor of 2. The result is then compared to the expected value of 5 using 'assertEquals'. A custom message is provided for the assertion failure: "10/2 did not yield 5".

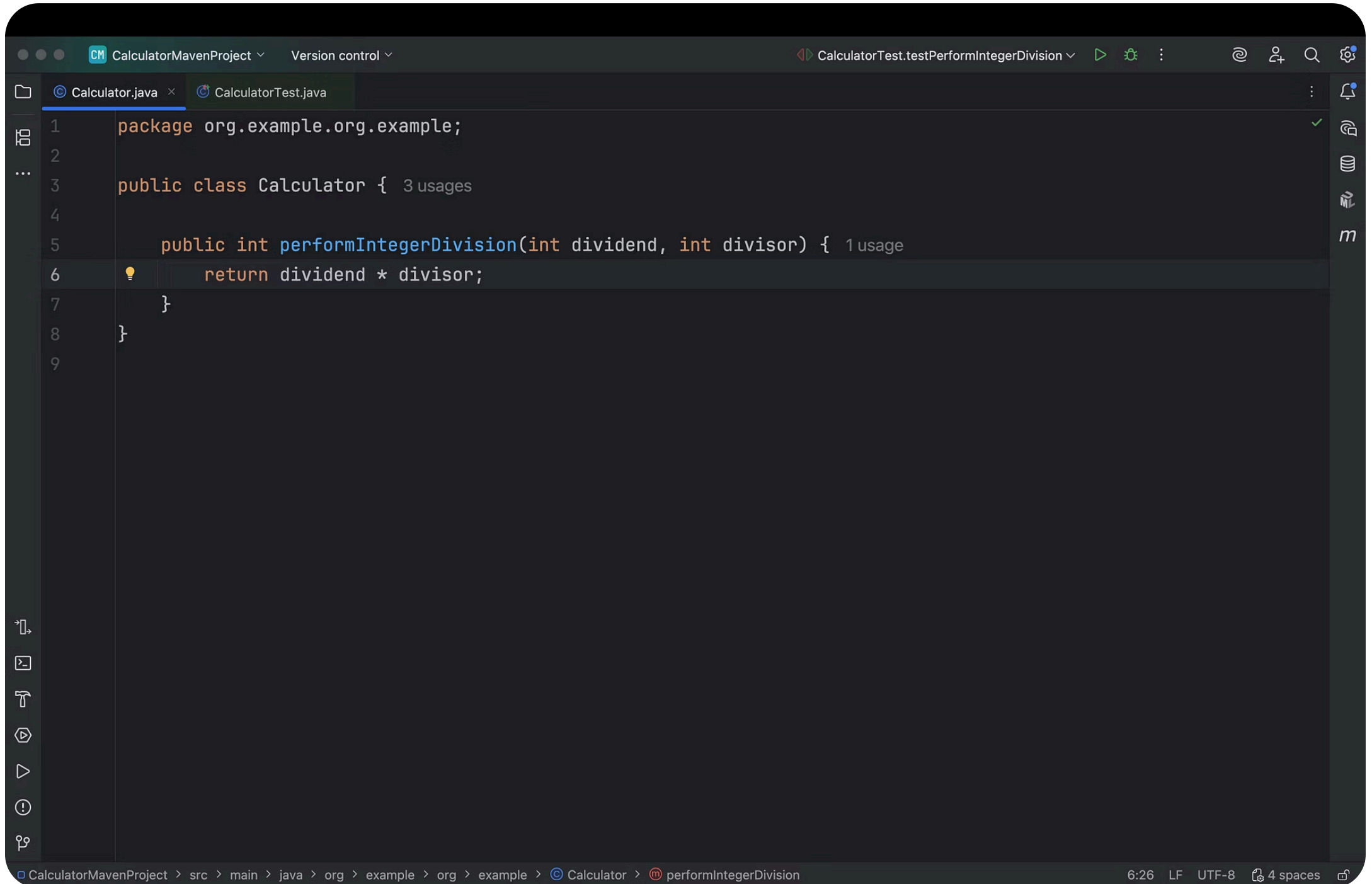
```
1 package org.example;
2
3 import org.example.org.example.Calculator;
4 import org.junit.jupiter.api.Test;
5
6 import static org.junit.jupiter.api.Assertions.assertEquals;
7
8 public class CalculatorTest {
9
10     @Test
11     public void testPerformIntegerDivision() {
12         Calculator calculator = new Calculator();
13         int result = calculator.performIntegerDivision( dividend: 10, divisor: 2);
14         assertEquals( expected: 5,result, message: "10/2 did not yield 5");
15     }
16 }
17
```

The bottom status bar shows the file path: 'CalculatorMavenProject > src > test > java > org > example > CalculatorTest > testPerformIntegerDivision'. The time is 14:56, and the encoding is UTF-8 with 4 spaces.

Assertions can accept an optional custom message to describe failure context

```
assertEquals(5,result, "10/2 did not yield 5");
```

Deliberately include a bug (* instead of /)

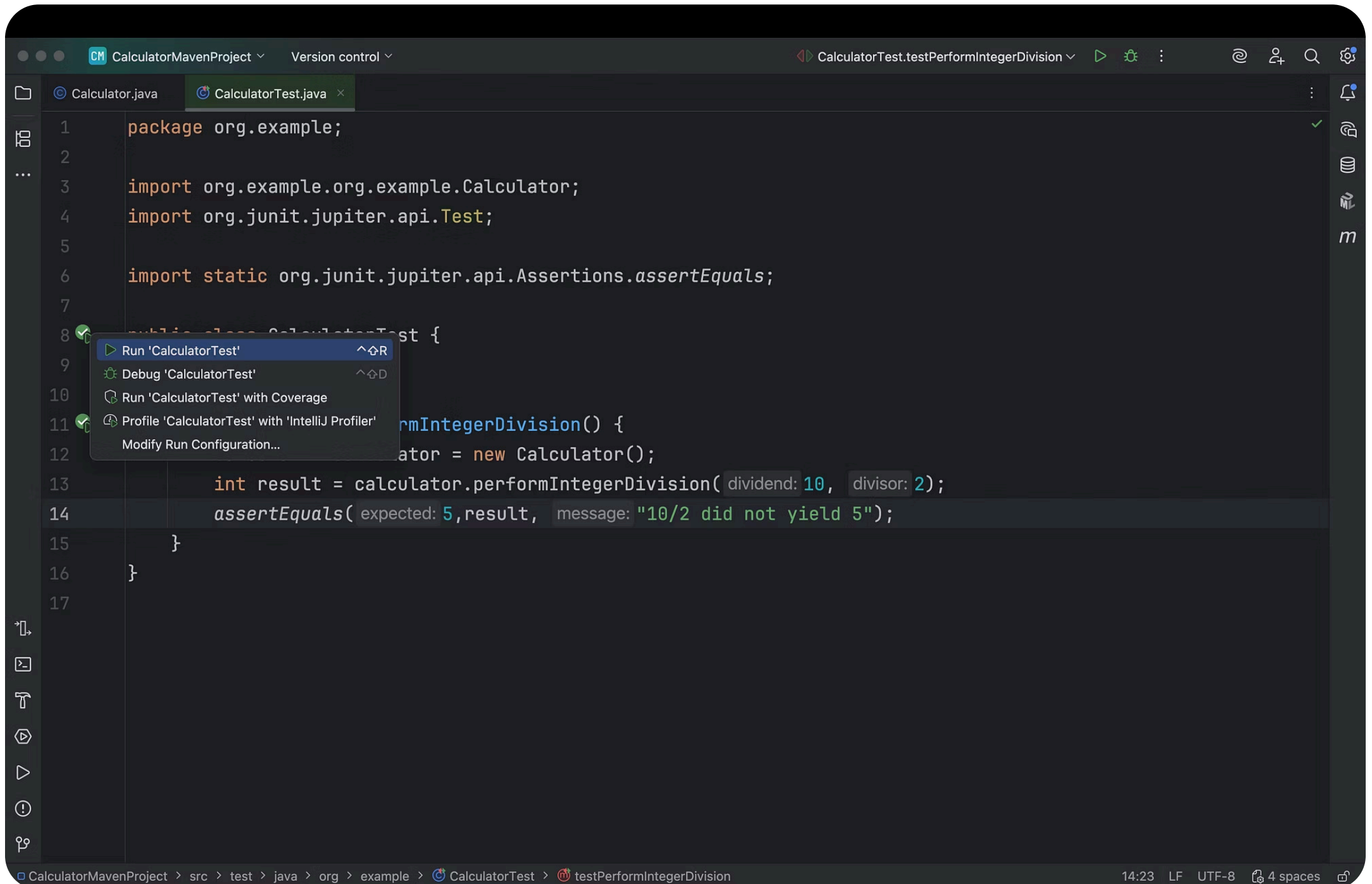


The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The 'Version control' tab is active. The file explorer on the left shows 'Calculator.java' and 'CalculatorTest.java'. The main editor displays the code for 'Calculator.java'. The code is as follows:

```
1 package org.example.org.example;
2
3 public class Calculator { 3 usages
4
5     public int performIntegerDivision(int dividend, int divisor) { 1 usage
6         return dividend * divisor;
7     }
8 }
9
```

The IDE interface includes a top bar with project and file names, a left sidebar with navigation icons, and a bottom status bar showing the current file path: 'CalculatorMavenProject > src > main > java > org > example > org > example > Calculator > performIntegerDivision'. The status bar also displays '6:26 LF UTF-8 4 spaces'.

Running the unit test to see it fail and produce a custom error message



Test failed

The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The editor displays the file 'CalculatorTest.java' with the following code:

```
8 public class CalculatorTest {  
11     public void testPerformIntegerDivision() {  
12         Calculator calculator = new Calculator();  
13         int result = calculator.performIntegerDivision(dividend: 10, divisor: 2);  
14         assertEquals(expected: 5, result, message: "10/2 did not yield 5");  
15     }  
16 }  
17
```

Below the editor, the 'Run' tab is active, showing the test 'CalculatorTest.testPerformIntegerDivision' has failed. The console output displays the following error:

```
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...  
  
org.opentest4j.AssertionFailedError: 10/2 did not yield 5 ==>  
Expected :5  
Actual   :20  
<Click to see difference>  
  
> <5 internal lines>  
    at org.example.CalculatorTest.testPerformIntegerDivision(CalculatorTest.java:14)
```

The bottom status bar indicates the file path 'CalculatorMavenProject > src > test > java > org > example > CalculatorTest' and the time '14:23'.

Notice the console output with the custom error message!

Fix the bug in the code (/ instead of *)

The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The main editor displays the 'Calculator.java' file with the following code:

```
1 package org.example.org.example;
2
3 public class Calculator { 3 usages
4
5     public int performIntegerDivision(int dividend, int divisor) { 1 usage
6         return dividend / divisor;
7     }
8 }
9
```

A red lightbulb icon is next to line 6, indicating a warning or error. The 'Run' tab at the bottom shows the test 'CalculatorTest.testPerformIntegerDivision' failed. The error message is:

```
org.opentest4j.AssertionFailedError: 10/2 did not yield 5 ==>
Expected :5
Actual   :20
<Click to see difference>

> <5 internal lines>
at org.example.CalculatorTest.testPerformIntegerDivision(CalculatorTest.java:14)
```

The status bar at the bottom shows the file path: 'CalculatorMavenProject > src > main > java > org > example > org > example > Calculator > performIntegerDivision'. The bottom right corner displays '6:26 LF UTF-8 4 spaces'.

Test passed!

The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The active file is 'CalculatorTest.java', which contains the following code:

```
8 public class CalculatorTest {  
11     public void testPerformIntegerDivision() {  
12         Calculator calculator = new Calculator();  
13         int result = calculator.performIntegerDivision(dividend: 10, divisor: 2);  
14         assertEquals(expected: 5, result, message: "10/2 did not yield 5");  
15     }  
16 }  
17
```

Below the code editor, the 'Run' tab is active, showing the execution of 'CalculatorTest.testPerformIntegerDivision'. The test passed successfully, as indicated by the green checkmark and the message '1 test passed 1 test total, 11 ms'. The output window shows the command path and the exit code:

```
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...  
  
Process finished with exit code 0
```

The bottom status bar shows the project path: 'CalculatorMavenProject > src > test > java > org > example > CalculatorTest > testPerformIntegerDivision'. The time is 14:56, and the encoding is UTF-8.



Improving Debugging with Better Assertions

Reduce Confusion

Custom messages reduce confusion when many assertions exist in a single test

Highlight Scenarios

They highlight the expected scenario and the problematic input values

Speed Up Debugging

Clear failure messages speed up debugging and code reviews

Communicate Intent

Encourages writing tests that communicate intent rather than relying solely on code

Benefit Reviewers

Developers reviewing reports benefit from extra context in failure output

Conclusion

- **Assertions** determine **test success** by **evaluating expected vs. actual results**
- Adding **custom messages** improves **clarity** and **accelerates debugging**
- Good **assertions** make **tests easier** to **understand, maintain, and review**
- Using **descriptive messages** is a simple yet powerful **testing best practice**

Open Floor Discussion



Your Practice

Do you usually include custom messages in your assertions?

Message Format

What kind of message format do you find most useful during debugging?

Level of Detail

How detailed should assertion messages be in large, complex projects?